Bachelor Thesis Midterm

A Multimodal XR Museum with vitrivr and Godot

Jannick Seper

University of Basel

22 July 2026

VIRTUE no longer matches today's media and XR landscape

01

Aged technology

The predecessor relies on Cineast and Cottontail DB.

02

Limited media scope

VIRTUE primarily supports images rather than heterogeneous collections.

03

New XR landscape

Modern headsets provide new interaction modes across multiple platforms.

Need: a modern XR museum for similarity-based exploration of multimodal collections.

Project idea: extend VIRTUE into a multimodal XR museum

PREDECESSOR

VIRTUE

- dynamically arranged image galleries in VR
- retrieval with spatial exploration
- Cineast and Cottontail DB
- image collections

→

THIS THESIS

Multimodal XR museum

- vitrivr-engine and pgvector
- images, videos, and 3D models
- multiple headsets through OpenXR
- similarity-driven exploration

Background: key terms

Term	Meaning	Role in this project
VR	**Virtual Reality**	The fully virtual museum environment experienced through a headset.
XR	**Extended Reality**	The umbrella term covering VR, augmented reality, and mixed reality.
OpenXR	**Cross-platform XR API**	A common interface for headsets, controllers, tracking, and XR runtimes.
CLIP	**Contrastive Language–Image Pre-training**	Maps text and images to comparable numerical embeddings.
HUD	**Head-up display**	Shows search progress and status messages within the user's view.

Three goals define the prototype

01
Retrieve

vitrivr-engine and pgvector provide semantic search over multimodal embeddings.

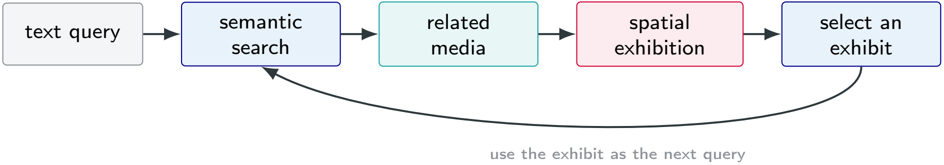
02
Experience

The XR application presents retrieved images, videos, and 3D models as interactive exhibits.

03
Explore

Users trigger new similarity searches from exhibits.

Similarity creates a continuous exploration loop



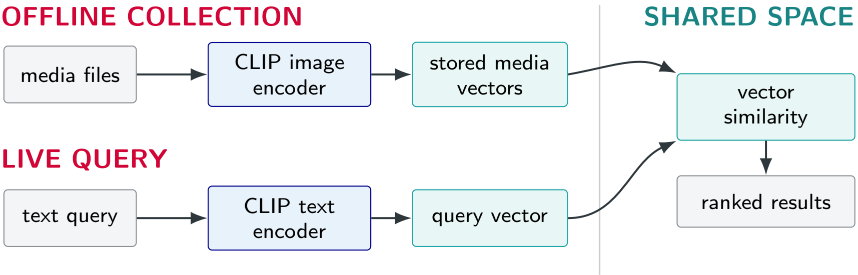
One concept

The interaction is independent of whether a result is an image, video, or 3D model.

Two entry points

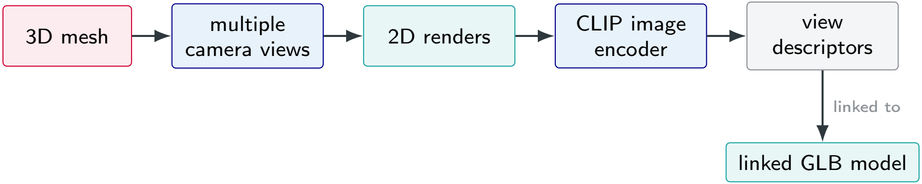
Exploration can start from language or continue directly from an existing exhibit.

CLIP creates a shared comparison space



Key distinction: the collection is encoded once; a live search only encodes the new query and compares it with the stored vectors.

Rendered views make 3D models searchable



Rather than encoding the mesh itself, the system searches over rendered views. A matching view points back to the original GLB model, which is then loaded into the museum.

Three responsibilities shape the system

01
XR experience

02
Application logic

03
Retrieval + media

XR client $\xrightarrow{\text{intent}}$ use cases $\xrightarrow{\text{query}}$ retrieval service

Interaction
input, navigation, selection

Presentation
menu, rooms, exhibits

Orchestration
validate, search, limit

Backend boundary
hides backend details

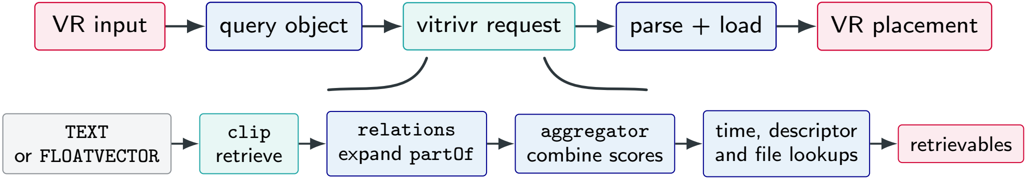
Semantic retrieval
embeddings, vector index

Delivery
local files or HTTP

Design decision: separate experience, orchestration, and retrieval so each can evolve independently.

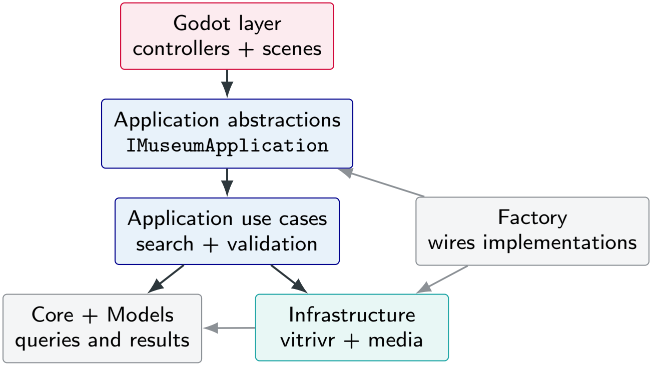
One vitrivr request supports both search modes

SIMPLIFIED REQUEST-RESPONSE FLOW



Only the input mapping changes. Response parsing, media loading, and VR placement are shared. The returned vector enables the next result-to-result search.

Godot and vitrivr remain replaceable



Why Godot?

01

Godot XR Tools

Ready-made building blocks for movement, interaction, grabbing, and XR interfaces.

02

OpenXR Vendors

Simple device integration for Quest and Focus; Windows uses the regular OpenXR runtime.

Controller and hand input use the same scenes and application logic.

Godot scenes separate runtime, state, and content

`Main`

10 direct child nodes, grouped by role

XR RUNTIME

`XrRenderProfile` `StartXR` `WorldEnvironment`

STATE + WIRING

`SearchSettingsStore` `GameSettingsStore`

`SearchUseCaseFactory` `PlatformSwitcher`

SCENE INSTANCES

`Player` → `XROrigin3D`, camera + HUD
hands + controllers

`MenuNode` → tutorial + keyboard
server + game settings

`MuseumNode` → search + room
media slots + 3D rooms

Entering the museum

app start → XR menu → validate server → optional tutorial → enter museum

`CanEnterMuseum` = server valid **and** (tutorial disabled **or** completed)

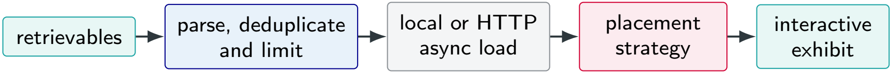
While in the menu

The museum is hidden, player movement is disabled, and configuration happens inside XR.

On transition

`PlatformSwitcher` moves the XR rig, swaps environment and HUD, and preserves museum state.

Media become exhibits



2D MEDIA

Images and videos use centred wall slots and aspect-aware frames.

3D MODELS

GLB objects are instanced on stands and fitted to their bounds.

Every exhibit stores its path, name, and CLIP vector for **Similar**. 3D models also support **Original Size**.

Current status

Backend & data

A1 Ingestion and extraction ★
Completed

A2 Similarity search integration ★
Completed

A3 Performance optimization ☆
Partial

VR application

B1 Scene generation ★
Completed

B2 OpenXR setup ★
Completed

B3 User interaction ★
Partial

B4 Advanced user interaction
Partial

B5 Gesture-based interaction ☆
Completed

Evaluation & documentation

D1 System evaluation ★
Partial

D2 Documentation ★
Missing

D3 User evaluation ☆
Missing

Demo

Search $\rightarrow$ explore $\rightarrow$ inspect $\rightarrow$ search again

Evaluation questionnaire

01
SUS

10 standard statements about usability.

02
Project questions

Navigation, controls, responsiveness, 3D interaction, and design.

03
Context + feedback

VR experience, device, usage mode, and one open question.

Next steps

1. Resolve slow media loading issues.
2. Integrate and test streaming to Apple Vision Pro.
3. Run user evaluations.
4. Complete the reproducible setup and documentation.
5. Finish the thesis.

TAKEAWAY

The prototype modernises the VIRTUE stack and already closes the loop from semantic query to spatial exploration.

Godot turns retrieval results into a device-aware XR experience.